

Trabajo Práctico — Semana 3

Base de Datos II — Unidad 2: Optimización de Consultas

Parte 2

2.2 Tabla de resultados (incluir en la entrega, una fila por consulta)

Consulta	Plan antes	Cambio aplicado	Plan después	Mejora esperada
Filtra y lista los pedidos consolidados que superan cierta cantidad de ítems y un monto facturado mínimo, ordenados por rentabilidad.	Utiliza un Merge Join masivo combinando un Nested Loop con las tablas de clientes y pedidos y un recorrido eficiente (Index Scan) sobre la clave primaria de detalle_pedido.	Mantenimiento de la estructura original y aprovechamiento directo de los índices primarios preexistentes (pedido_pkey, cliente_pkey, pk_detalle_pedido).	Limit ejecutándose sobre un Sort y GroupAggregate con recorridos indexados eficientes en las fuentes principales.	Estabilizar los tiempos de agregación a nivel global sin incurrir en lecturas secuenciales innecesarias en las tablas principales.
Obtiene de forma directa todos los productos y cantidades pertenecientes a un pedido en particular.	Ejecutaba un Bitmap Heap Scan apoyándose en la clave primaria compuesta existente (pk_detalle_pedido), dado que id_pedido es la primera columna de dicha clave.	Creación y uso de un índice simple dedicado exclusivamente sobre la columna de búsqueda (CREATE INDEX idx_detalle_pedido ON detalle_pedido (id_pedido);).	Utiliza un Bitmap Index Scan sobre el nuevo índice específico (idx_detalle_pedido_id_pedido) seguido de un Bitmap Heap Scan acotado al bloque físico exacto.	Mantener el acceso rápido y optimizado para consultas acotadas por ID de pedido con un costo mínimo de I/O en disco.
Encuentra todas las líneas de detalle donde se haya vendido un producto determinado en todo el historial de la tienda.	Al no ser id_producto la primera columna de la clave primaria compuesta, el optimizador realizaba un costoso Parallel Seq Scan recorriendo secuencialmente las 700,000+ filas de detalle_pedido.	Creación de un índice dedicado para la segunda columna de la relación (CREATE INDEX idx_detalle_pedido_id_producto ON detalle_pedido (id_producto);).	Cambia radicalmente a un Index Scan directo utilizando idx_detalle_pedido_id_producto.	Eliminar por completo el escaneo secuencial masivo de la tabla, reduciendo el tiempo de ejecución de ~57 ms a menos de 0.15 ms (una mejora drástica de velocidad para filtrados por producto).

Parte 3

Afirmación	¿Correcta?	Corrección / Evidencia
"Index Scan using idx_detalle_pedido_id_producto on detalle_pedido. El motor de PostgreSQL usó el índice idx_detalle_pedido_id_producto para buscar directamente en el árbol B ordenado por id_producto. Localizó las 14 filas que coinciden con el valor 2, y por cada una fue a buscar el resto de los campos (id_pedido, cantidad, precio_unitario_facturado) a la tabla física detalle_pedido. No tuvo que recorrer nada más de la tabla."	Correcta	Descripción precisa del comportamiento de un Index Scan.
"Index Cond: (id_producto = 2). Es la condición que el motor aplicó dentro del índice para filtrar. Solo recorre las ramas del árbol que contienen el valor 2, ignorando todo lo demás."	Correcta	Interpretación correcta de la condición aplicada en el índice.
"Buffers: shared hit=7. Significa que el motor leyó 7 bloques de memoria compartida (caché del servidor) para completar toda la operación. Cada bloque son 8 KB, así que en total leyó ~56 KB. Para una tabla de 700,000+ filas, es una cantidad mínima de lectura."	Correcta	Cálculo correcto (7 bloques × 8 KB = 56 KB).
Planning Time: 3.974 ms. El tiempo que el	Correcta	Interpretación correcta del tiempo de

optimizador tardó en decidir el plan. Es mayor que el tiempo de ejecución mismo, porque PostgreSQL tuvo que consultar las estadísticas de la tabla y del índice para elegir la mejor ruta."		planificación y su relación con el tiempo de ejecución.
"Execution Time: 0.174 ms. El tiempo real que la consulta tardó en ejecutarse. 0.174 milisegundos. Sin el índice, esta misma consulta tardaba ~57 ms porque escaneaba las 700,000 filas completas con un Parallel Seq Scan. La mejora es de aproximadamente 328 veces más rápido."	Parcialmente correcta	La parte de Execution Time es correcta. La comparación con el escenario sin índice (~57 ms, Parallel Seq Scan, mejora de 328×) no está en el plan y no puede validarse con la evidencia proporcionada. Es una afirmación externa que, aunque posiblemente cierta, no se deriva del EXPLAIN entregado.

Parte 5

Nombre del Equipo	Equipo A	Equipo B
Estrategia aplicada	Índice B-tree parcial sobre precio_lista con filtro WHERE activo = TRUE.	Índice covering (INCLUDE id_categoria, nombre, stock) para evitar acceso al heap.
Tiempo Query Original	10.87 ms.	
Tiempo Query Optimizada	6.00 ms.	2.33 ms.
Mejora	45% más rápida. El índice permite un Bitmap Index Scan que reduce las filas escaneadas de 50k a 5k antes del join.	79% más rápida. El Index Only Scan lee todo directamente del índice sin tocar la tabla física.

DUIA

Herramienta	Para qué se usó	Prompt / spec (resumen)	Se aceptó / se descartó — por qué
OpenCode	De programación nos dieron un harness que modifica el flujo de trabajo en opencode, volviéndolo más estratégico y siendo capaz de no solo resolver de manera agentica, sino de planificar. Así que se usó tanto para la planificación de todo el desarrollo, como para concretar el mismo.	Se utilizaron prompts estructurados (objetivo-contexto-restricciones-salida) para mantener un control de lo que la ia debe y no hacer, y cómo hacerlo. Estos fueron desde analizar errores en los archivos sql (como en el schema), hasta crear nuevos sql (carga masiva de datos, consultas query, specs).	La IA tuvo errores al crear la carga masiva de datos(lentitud en la carga)y planteó erróneamente las queries (consultas que no necesitaban optimización por índice). Estos errores fueron corregidos modificando los prompts y verificando activamente que todo funcionara. La mayoría de cosas se mantuvieron como las creo.